A Lazy Decision Procedure for LIA^* in cvc5, Explained by Example

Claude (Anthropic)

June 2026

Abstract

This document explains the algorithms behind the lazy decision procedure for the *additive closure* (star) operator in cvc5, and the sequence of improvements that took a benchmark family from 450/480 solved to 480/480 (median 45 ms per instance). It is written to be self-contained: every concept is defined in plain language when it first appears, every definition and every claim comes with a small worked example placed right where it is used, and one *running example* is carried through the whole document. Negative experimental results are included, because two of them supplied the machinery for the final breakthroughs.

1 The problem

1.1 Vectors, sums, and the star of a set

We work with vectors of non-negative integers, written $\vec{x} = (x_1, \ldots, x_n)$. Vectors are added coordinate-wise: $(1,0) + (0,2) = (1,2)$.

Running example (used throughout this document). Let p be the predicate

$$p(x,y) \;=\; \big(\underbrace{x \geq 1 \wedge y = 0}_{\text{“horizontal”}}\big) \;\vee\; \big(\underbrace{x = 0 \wedge y \geq 2}_{\text{“vertical”}}\big),$$

and let S be the set of non-negative integer vectors satisfying it:

$$S \;=\; \{(1,0),\,(2,0),\,(3,0),\,\ldots\} \;\cup\; \{(0,2),\,(0,3),\,(0,4),\ldots\}.$$

The *star* of S, written S^*, is the set of all vectors obtainable by adding finitely many members of S (taking *zero* members is allowed, which produces the zero vector $\vec{0}$):

$$S^* \;=\; \Big\{\, \vec{s}_1 + \cdots + \vec{s}_k \;\Big|\; k \geq 0,\; \vec{s}_j \in S \,\Big\}.$$

Example. In the running example:

- $(3,7) \in S^*$, because $(3,7) = (3,0) + (0,7)$, and both summands satisfy p.
- $(0,0) \in S^*$: it is the empty sum. Note $(0,0) \notin S$ itself — the star always contains $\vec{0}$ even when the set does not.
- $(0,1) \notin S^*$: every member of S with a nonzero second coordinate has $y \geq 2$, and

coordinates never decrease when adding non-negative vectors, so no sum can reach $y = 1$.

So $S^* = \{(0,0)\} \cup \{(x,0) \mid x \geq 1\} \cup \{(0,y) \mid y \geq 2\} \cup \{(x,y) \mid x \geq 1,\, y \geq 2\}$.

1.2 The `int.star-contains` literal, and where it comes from

cvc5 accepts the operator

$$\ell \;=\; \mathit{int.star\text{-}contains}\big(\lambda x\, y.\; p(x,y),\;\; v_1,\, v_2\big),$$

a formula that is true exactly when the vector $\vec{v} = (v_1, v_2)$, built from the integer terms v_1, v_2 of the input problem, belongs to S^*. The first argument is a *lambda term*: an anonymous function binding the variables x, y of the predicate, so that the predicate can be passed around as a value.

Example (where such constraints come from). Consider sets A, B over some universe and the constraint $|A| = 3 \wedge |A \cap B| = 0$. Every element of the universe falls into exactly one of four *regions* (in A only, in B only, in both, in neither). Summing, per region, the 0/1 indicator vector of each element shows that the vector of region cardinalities is a *sum of per-element vectors*, each satisfying a fixed predicate — i.e. a member of a star set. This is how Boolean algebra with Presburger arithmetic (BAPA, sets with cardinalities) and its multiset variant (MAPA) are encoded; our benchmarks come from such encodings, where the predicates contain many ite (if-then-else) terms defining indicator values.

Deciding ℓ means answering: *can $\vec{v}$ be written as a sum of vectors that each satisfy p?* The rest of this document is about doing that inside an SMT solver.

1.3 SMT vocabulary used in this document

For readers from neighbouring areas we fix the few solver terms we need. An SMT solver couples a SAT solver (handling the Boolean structure) with *theory solvers* (handling, e.g., linear integer arithmetic, LIA). A *lemma* is a formula a theory solver hands to the SAT solver to be conjoined with the problem; it must not change satisfiability.

Example (lemma). The arithmetic solver may emit $x \geq 1 \Rightarrow x \geq 0$ — valid, so always safe. A lemma may also introduce *fresh* symbols: $d \Rightarrow x = 2$ with a brand-new Boolean d is safe even though it is not valid, because any model can be extended by choosing $d = \mathit{false}$. Such lemmas are called *satisfiability-preserving*.

A *candidate model* is the assignment the solver currently believes satisfies everything; theory solvers inspect it at *full effort* (when the Boolean search has assigned all variables) and either accept it or emit new lemmas. The SAT solver maintains a *trail* of variable assignments; each is either a *decision* (a guess) or a *propagation* (forced by a clause). The number of decisions on the trail is the *decision level*; anything implied with no decisions at all is *fixed at level 0* — it holds in every model.

Example (trail, propagation, decision level). Clauses: $(a \vee b)$, $(\neg a \vee c)$. The solver *decides* $a = \mathit{true}$ (level 1); the second clause then *propagates* $c = \mathit{true}$ at level 1. Nothing here is fixed at level 0: a different decision ($a = \mathit{false}$) is still possible. If instead the input contained the unit clause (a), then a would be fixed at level 0.

Finally, a *subsolver* is a separate, private SMT solver instance that a theory creates for its own queries; *incremental* means assertions can be added between satisfiability checks; and checking with

assumptions means temporarily assuming extra literals for a single check, without asserting them permanently.

Example (assumptions and unsat core). Assert $x \geq 1$ permanently; then ask `checkSat` under assumptions $\{x \leq 0,\ y \geq 5\}$. The answer is unsat, and the solver can report *which assumptions were actually used*: the core $\{x \leq 0\}$. The assumption $y \geq 5$ played no role. This "unsat-assumptions core" feature is used twice in this document.

2 The reduction to linear arithmetic

2.1 Cells: the predicate in disjunctive normal form

A formula is in *disjunctive normal form* (DNF) if it is a disjunction (or-list) of conjunctions (and-lists) of atomic constraints. Each conjunction of linear constraints describes a convex polyhedron; we call the polyhedra of the predicate's DNF its *cells*.

Example. The running example $p(x, y) = (x \geq 1 \wedge y = 0) \vee (x = 0 \wedge y \geq 2)$ is already in DNF and has two cells: the horizontal ray and the vertical ray. A predicate that is *not* in DNF, such as $(x \geq 1 \vee y \geq 1) \wedge (x \leq 5)$, is converted by distributing: $(x \geq 1 \wedge x \leq 5) \vee (y \geq 1 \wedge x \leq 5)$ — two cells.

Example (why ite terms blow the DNF up). The equation $u = \mathsf{ite}(x \geq 1,\ 1,\ 0)$ ("u is 1 if $x \geq 1$, else 0") becomes the disjunction $(x \geq 1 \wedge u = 1) \vee (x \leq 0 \wedge u = 0)$: one ite doubles the number of cells. Two independent ites give $2 \times 2 = 4$ cells, and the hard benchmark predicates contain up to *23* ites — up to $2^{23} \approx 8.4$ million potential cells. This is why the *eager* strategy below fails and a lazy one is needed.

2.2 Describing one cell: generators and rays

To turn a cell into arithmetic, we need a finite description of its infinitely many integer points. The library Normaliz provides one: a finite set of *base points* g (called module generators) and a finite set of *directions* $\vec{h}$ (the Hilbert basis of the cell's *recession cone* — the set of directions in which one can walk forever without leaving the cell). Every integer point of the cell is $g + \sum_k l_k \vec{h}_k$ for some base point and non-negative integers l_k.

Example. The horizontal cell $x \geq 1 \wedge y = 0$ has one base point $g = (1, 0)$ and one direction $\vec{h} = (1, 0)$: its points are $(1, 0) + l \cdot (1, 0) = (1{+}l,\ 0)$, i.e. $(1, 0), (2, 0), (3, 0), \ldots$ The vertical cell has base point $(0, 2)$ and direction $(0, 1)$.

A two-direction example: the cell $x \geq y \wedge y \geq 0$ (a quarter-plane tilted onto the diagonal) has base point $(0, 0)$ and Hilbert basis $\{(1, 0),\ (1, 1)\}$; e.g. $(3, 2) = (0, 0) + 1 \cdot (1, 0) + 2 \cdot (1, 1)$. Note $(1, 1)$ is needed: it cannot be built from $(1, 0)$ and any other single direction inside the cell.

2.3 The star constraints

How do we express "$\vec{v}$ is a sum of *several* points of a cell"? Take the cell's base point g with a multiplier $\mu \geq 0$ meaning *how many summands* are drawn from this cell, and ray multipliers $l_k \geq 0$ accumulating all the ray steps of all those summands; the cell contributes $\mu \cdot g + \sum_k l_k \vec{h}_k$. The side

condition $\mu = 0 \Rightarrow l_k = 0$ says rays are only available if at least one summand was actually taken. Summing the contributions of all cells and equating with $\vec{v}$ gives the *star constraints*.

Example. For the running example ($p(x, y) = (x \geq 1 \wedge y = 0) \vee (x = 0 \wedge y \geq 2)$) the star constraints over the fresh integer unknowns μ_1, l_1, μ_2, l_2 are

$$v_1 = \mu_1 \cdot 1 + l_1 \cdot 1, \qquad v_2 = \mu_2 \cdot 2 + l_2 \cdot 1, \qquad \mu_i, l_i \geq 0, \qquad \mu_i = 0 \Rightarrow l_i = 0.$$

$\vec{v} \in S^*$ *iff* these constraints are satisfiable:

- $\vec{v} = (3, 7)$: take $\mu_1 = 1, l_1 = 2$ (one horizontal summand, stretched by 2) and $\mu_2 = 1, l_2 = 5$. Satisfiable — and indeed $(3, 7) = (3, 0) + (0, 7) \in S^*$.
- $\vec{v} = (0, 1)$: $v_2 = 1$ forces $\mu_2 = 0$ (one vertical summand already contributes 2), hence $l_2 = 0$, hence $v_2 = 0 \neq 1$. Unsatisfiable — and indeed $(0, 1) \notin S^*$.

The procedure can therefore emit the single lemma $\ell \Leftrightarrow \mathit{star}$ and let the ordinary LIA engine decide everything. The *eager* strategy does exactly this, building all cells up front — and dies on the 2^{23}-cell predicates, as explained above. Everything from here on is about avoiding the full DNF.

3 The lazy loop

The lazy strategy discovers cells *one at a time*, maintaining an *under-approximation*: a version of the star constraints, star_k, built from only the k cells found so far. Since missing cells can only *remove* ways of summing, star_k describes a *subset* of S^*.

The discovery engine is a subsolver holding

$$\mathit{base}(\vec{y}) \;=\; p(\vec{y}) \wedge \vec{y} \geq 0$$

over fresh constants $\vec{y} = (y_1, \ldots, y_n)$. (Fresh constants are needed because a subsolver cannot be given a formula with the lambda's bound variables; replacing bound variables by fresh constants is standard and preserves satisfiability — e.g. asking whether $\lambda x.\, x{\geq}1$ is satisfiable becomes asking whether $y \geq 1$ is, for a new symbol y.)

Each *round* of the loop does the following.

1. Run `checkSat` on the subsolver, which also holds the negations of all previously found cells. If unsat, every cell is covered: emit the *exact* lemma $\ell \Leftrightarrow \mathit{star}$ and stop. (This exactness is what makes unsat answers for ℓ possible at all.)
2. Otherwise read the subsolver's model and extract the cell containing it, by fixing every atomic constraint of *base* to the truth value it has in the model.
3. Hand the cell to Normaliz, extend the star constraints to star_k, and emit the *guarded* lemma $g_k \Rightarrow (\ell \Leftrightarrow \mathit{star}_k)$, where g_k is a fresh Boolean. Also emit $\neg g_{k-1}$, retiring the previous round's guard. The solver is told to *prefer* $g_k = \mathit{true}$ when it gets to choose (a *phase preference*: a hint for decisions, never a constraint).
4. Assert the negation of the new cell in the subsolver, so the next round's model lies somewhere new.

Example (two rounds on the running example). Round 1: the subsolver holds *base* only; say its model is $\vec{y} = (4, 0)$. The atoms of *base* get the values $x{\geq}1 \mapsto \mathit{true}$, $y{=}0 \mapsto \mathit{true}$, $x{=}0 \mapsto \mathit{false}$, $y{\geq}2 \mapsto \mathit{false}$, giving the cell $D_1 = (x \geq 1 \wedge y = 0 \wedge \neg(x{=}0) \wedge y < 2)$ — the horizontal cell, plus two redundant conjuncts (cleaned up in the next section). We emit $g_1 \Rightarrow (\ell \Leftrightarrow \mathit{star}_1)$ where star_1 is $v_1 = \mu_1 + l_1 \wedge v_2 = 0 \wedge \ldots$, and assert $\neg D_1$ in the subsolver. Round 2: the subsolver's model must avoid D_1, say $\vec{y} = (0, 2)$; we discover the vertical cell, emit $\neg g_1$ and $g_2 \Rightarrow (\ell \Leftrightarrow \mathit{star}_2)$, and the subsolver becomes `unsat` in round 3: enumeration complete.

Why this step is needed. Why guard the lemma instead of asserting $\ell \Leftrightarrow \mathit{star}_k$ outright? Because star_k is only an under-approximation. Concretely: after round 1 above, star_1 forces $v_2 = 0$, so for an input where $\vec{v} = (0, 5)$ the unguarded equivalence would force $\ell = \mathit{false}$ — but $(0, 5) \in S^*$ via the vertical cell, so on an input asserting ℓ the solver would wrongly answer `unsat`. With the guard, the SAT solver escapes by setting $g_1 = \mathit{false}$ (the lemma $g_1 \Rightarrow \cdots$ is then trivially satisfied), and the loop refines further. In the other direction no guard is needed: if the candidate model satisfies $p[\vec{v}]$ ($\vec{v}$ itself is a one-summand member) or satisfies star_k (a k-cell sum exists), then $\vec{v} \in S^*$ really holds — an under-approximation can never wrongly certify `sat`. The procedure accepts a model exactly in these two cases (the *model-value shortcut*).

4 Four refinement strategies, and what they taught us

This section reports engineering variants of the loop. Two are negative results; we keep them because their machinery returns later, and because they redirected the investigation to the real bottlenecks.

4.1 Push/pop refinement

In the baseline, the subsolver accumulates one negated cell per round: after three rounds it holds $\mathit{base}, \neg D_1, \neg D_2, \neg D_3$, and these assertions live forever. The push/pop variant instead keeps a single combined formula: since $\neg(D_1 \vee D_2 \vee D_3)$ implies $\neg(D_1 \vee D_2)$, each round can *pop* (retract) the previous combined formula and assert the new one, keeping exactly two live assertions — the hope being lower memory in the subsolver.

Measured result: no gain. On the hardest instance, both variants time out with the *same* ∼3.2 GB — profiling showed the memory lived in the *main* solver's accumulated lemmas, not in the subsolver. On the full suite, push/pop solves slightly *fewer* (460 vs. 463 of 480): popping throws away everything the subsolver had *learned* about already-covered regions, and the cumulative formula must be re-processed from scratch each round. **Lesson: measure the bottleneck before optimizing it.**

4.2 Enumerating inside the main solver

A more radical variant deletes the subsolver and lets the *main* search enumerate cells: the fresh constants $\vec{y}$ are added to the main problem, constrained by lemmas $h_0 \Rightarrow \mathit{base}(\vec{y})$ and, per discovered cell, $h_k \Rightarrow h_{k-1}$ and $h_k \Rightarrow \neg D_k(\vec{y})$, where the h_k are fresh Booleans (so activating h_k places $\vec{y}$ in a not-yet-covered cell). When the candidate model sets $h_k = \mathit{true}$, the values of $\vec{y}$ are read off and a new cell is harvested.

Why this step is needed. Three problems appeared, each with a concrete failure.

(1) Detecting completeness. In the subsolver version, "unsat" announces that all cells are covered. Here one would like to wait until the SAT solver proves $\neg h_k$ outright — but a SAT solver only derives what the search forces it to derive, and nothing forces it to keep trying $h_k = \mathit{true}$; runs livelocked. The repair is the *driver lemma* $\ell \Rightarrow (p[\vec{v}] \vee \mathit{star}_k \vee h_k)$: if the input asserts ℓ and the model certifies it by neither $p[\vec{v}]$ nor star_k, the model *must* set $h_k = \mathit{true}$, i.e. exhibit a fresh cell. When all cells are covered, the h_k branch becomes contradictory and only the certified branches survive.

(2) The empty sum. The first-round driver must not be $\ell \Rightarrow (p[\vec{v}] \vee h_0)$. Counterexample: take the predicate $p(x) = x \geq 1 \wedge x \leq 0$, which no point satisfies, so $S^* = \{\vec{0}\}$; and an input asserting ℓ plus $v_1 = 0$. Then ℓ is *true* ($\vec{v} = \vec{0}$ is the empty sum), but $p[\vec{v}]$ is false and h_0 is contradictory (*base* is unsatisfiable) — the driver would force $\neg\ell$: a wrong unsat. The star branch must therefore always include the empty sum, $\vec{v} = \vec{0}$. This bug was found by an adversarial review and is now the regression test `empty_pred_sat.smt2`.

(3) Solver pops. In incremental usage (`push`/`pop` by the *user*), lemmas are retracted on pop but the extension's bookkeeping survives, so the enumeration constraints silently vanished (observed as a wrong sat). Fix: re-submit all enumeration lemmas every round; the solver's lemma cache drops duplicates for free and re-sends them after a pop.

Measured result: sound after the fixes, but clearly worse (447 vs. 463): every refinement round now drags the entire input problem along, and only one cell is harvested per full-effort check. The lasting value is the *decoupling lesson* — enumeration and the main search starve each other when coupled — which returns with the sign flipped in Section 5.

4.3 Generalizing the discovered cells (now the default)

Step 2 of the lazy loop fixes *every* atom of *base* to its model value. That makes cells as small as possible — bad, because more cells means more rounds.

Example. Recall from the round-1 walk-through that the model $\vec{y} = (4, 0)$ produced

$$D_1 \;=\; x \geq 1 \;\wedge\; y = 0 \;\wedge\; \neg(x = 0) \;\wedge\; y < 2.$$

The conjuncts $\neg(x = 0)$ and $y < 2$ come from atoms of the *other* cell and are redundant here. Now imagine the predicate also contained an unrelated ite over a variable w: its atoms (say $w \geq 5$ and its negation) would also be fixed, splitting the one horizontal cell into *two* cells ($w \geq 5$ and $w < 5$) that the loop must discover separately. With m irrelevant atoms, one logical cell shatters into up to 2^m discovered cells.

Boolean generalization repairs this with a *prime implicant* computation. (An implicant of a formula is a conjunction of literals that forces the formula to be true; it is *prime* if dropping any literal breaks that. Example: for $a \wedge (b \vee c)$, the conjunction $a \wedge b \wedge c$ is an implicant; $a \wedge b$ is a prime implicant.) The algorithm walks over the kept literals and, for each, temporarily marks its atom *unknown* and re-evaluates *base* in three-valued logic — where $\wedge$ yields *false* if any conjunct is *false*, *unknown* if any is unknown (and none *false*), and *true* otherwise; $\vee$ is the mirror image. If the formula still evaluates to *true*, every possible value of that atom keeps *base* true, so the literal is dropped for good.

Example. On D_1 above, with *base* $= (x{\geq}1 \wedge y{=}0) \vee (x{=}0 \wedge y{\geq}2)$: marking $\neg(x{=}0)$'s atom unknown leaves the first disjunct $\mathit{true} \wedge \mathit{true} = \mathit{true}$, so the disjunction is *true* — drop it. Marking $y < 2$'s atom ($y \geq 2$) unknown: same — drop it. Marking $x \geq 1$ unknown: the first disjunct becomes *unknown*, the second is *false*, so the disjunction is *unknown* — keep it. Result: $D_1' = x \geq 1 \wedge y = 0$, exactly the logical cell.

Soundness: three-valued *true* under a partial assignment means *every* completion of the unknowns satisfies *base* — i.e. the kept conjunction still implies *base*, so the cell's points all lie in S and the star constraints built from it never overshoot. The model point stays inside the (larger) cell, so asserting its negation still makes progress. And if two generalized cells overlap, nothing breaks: the correctness of the cell-by-cell star encoding only needs the cells to *cover* S, not to be disjoint — a sum grouped by cells stays a sum.

Measured result: the best general-purpose improvement of the whole project — discovered cells halve in size, 466/480 solved, 4.4× faster on the commonly solved instances. Enabled by default.

Semantic generalization (optional, off) pushes further: drop a literal whenever the rest *arithmetically* implies *base*, which the propositional view cannot see.

Example. *base* $= x \geq 1 \wedge x + y \geq 1 \wedge y \geq 0$, model $(1, 0)$. All three atoms are top-level conjuncts, so the propositional check keeps all three. But $x \geq 1 \wedge y \geq 0$ already implies $x+y \geq 1$ *as arithmetic*. The implication is verified with a probe subsolver holding $\neg$*base*: check, under assumptions $\{x \geq 1,\ y \geq 0\}$ — `unsat` confirms the implication, so $x + y \geq 1$ is dropped and the cell fattens.

Measured result: negative on this suite (465 solved, 2.25× slower) — each probe costs a solver call, and fatter cells have larger Hilbert bases, hence more multipliers. Kept as an option; its *probe-validation* pattern ("a claim about all of *base* = one `unsat` answer") is reused by the cut synthesis in Section 6.

4.4 Constant-size lemmas via partial sums

Round k's guarded lemma contains the sum constraints over *all* k cells, so the total text emitted over K rounds grows like K^2. One can instead define *running totals* once per round — $P_k = P_{k-1} +$ (round k's contribution), over fresh unknowns P_k — and shrink the guarded lemma to the constant-size $\vec{v} = P_k$. The definitions are safe to assert unconditionally: they only constrain fresh symbols (set all multipliers to 0 and every P_k to $\vec{0}$ to satisfy them in any model).

Measured result: clearly negative — memory rose (4.4 vs. 3.3 GB on the hard instance) and 13 fewer instances were solved. Two measured reasons. First, the per-cell machinery (multiplier bounds, coupling implications, definition rows) is itself tens of lemmas per round; the quadratic term we removed was not the dominant cost. Second, a subtle search effect: when the solver explores $\ell = \mathit{false}$ under the guarded equivalence, it must falsify the star formula; falsifying the old "fat" star was cheap (violate any one $\mu \geq 0$), but falsifying $\vec{v} = P_k$ means asserting a *disequality* $v_i \neq P_k[i]$, which integer reasoning handles by case-splitting ($<$ or $>$) — the count of those splits exploded (14 → 663). **Lesson: lemma *shape* affects search, not just lemma size.** This negative result redirected attention from memory to the two real bottlenecks, treated next.

5 Hard satisfiable instances: aim the search, then finish the job

The hardest `sat` instances (the `fol_0000099--106` family) survived everything above; the flagship `fol_0000100` timed out at 600 s in every configuration, running ∼1000 refinement rounds. Three

measured facts explain it, and each dictates one component of the cure.

Fact 1: a one-summand witness does not exist. The input's side constraints force $\vec{v}$ to be a sum of at least two members (checked standalone: side constraints $\wedge\ p[\vec{v}]$ is unsat), so the model-value shortcut cannot fire early; the star is unavoidable.

Fact 2: undirected discovery wanders. The subsolver's model is *any* point not yet covered — nothing relates it to $\vec{v}$. Traced on `fol_0000100`: the cells needed to decompose $\vec{v}$ appear within ∼30 rounds when the search is aimed (below), while the unaimed search ran 580 cones without assembling them (proved by an unsat-core analysis in the next section).

Fact 3: the endgame is starved. Even with the right cells present, certifying sat means the *main* solver must find integer values for hundreds of multipliers — an integer program. Integer reasoning in cvc5 finishes with *branch-and-bound*: if the relaxed (fractional) solution has $\mu = 1.5$, the solver splits on $\mu \leq 1 \vee \mu \geq 2$ and recurses. Two measured interferences kept this from ever completing: (i) the star extension runs *before* branch-and-bound each round, sees a fractional candidate (which fails the model-value shortcut), and immediately harvests another cell — rewriting and growing the integer program before the splitting could finish; (ii) the star's many fresh equality atoms enter the SAT solver with default (false-leaning) phases, so some atom is assigned *false* early, which *unit-propagates* the guard g_k to *false* long before any decision on g_k — measured: across 643 rounds the guard was never *true* even once, so the solver never even attempted the star. A phase *preference* on g_k is powerless against propagation.

The cure has three cooperating parts.

5.1 Part 1: guided enumeration (default on)

Proposition 1. *If $\vec{v} = \vec{s}_1 + \cdots + \vec{s}_k$ with all $\vec{s}_j \geq 0$, then every summand satisfies $\vec{s}_j \leq \vec{v}$ coordinate-wise.*

Proof. Removing non-negative summands never increases any coordinate. □

So only cells intersecting the box $\vec{y} \leq \vec{v}$ can contribute to a decomposition of $\vec{v}$. The subsolver query is therefore first asked under the assumptions $\{\, y_i \leq \hat{v}_i \,\}$, where $\hat{\vec{v}}$ is the value of $\vec{v}$ in the current candidate model; if the assumed query is unsat, it is retried without assumptions. The fallback keeps the completeness signal intact: "unsat without assumptions" still means full coverage.

Example. Running example, $p(x, y) = (x \geq 1 \wedge y = 0) \vee (x = 0 \wedge y \geq 2)$, with the input forcing $\vec{v} = (5, 0)$. The assumptions are $y_1 \leq 5 \wedge y_2 \leq 0$. The vertical cell needs $y_2 \geq 2$ — excluded outright. The enumeration will only ever look at the horizontal cell: the single cell that can decompose $(5, 0)$. On `fol_0000100`, 7 of the 14 indicator coordinates of $\vec{v}$ are 0, so the assumptions pin those y_i to 0 and the search space collapses onto the relevant region.

5.2 Part 2: a decoupled endgame (default on, every 10 cells)

Every N newly discovered cells, the extension builds a *fresh* subsolver and asks one clean question:

$$F_0 \ \wedge\ star_k \ \stackrel{?}{=}\ \mathsf{sat},$$

where F_0 is the set of arithmetic facts *fixed at decision level 0* in the main solver (recall the trail example: level-0 facts hold in every model — they are exactly the input-entailed constraints). A fresh solver has no history — no retired guards, no unlucky phases, no half-grown integer program — and runs branch-and-bound to completion in milliseconds. If the answer is sat, its model is a complete witness: concrete values for $\vec{v}$ and for every multiplier.

Why this step is needed. Both design choices were forced by failures.

Why only level-0 facts? The first version asserted *all* facts currently on the trail. On `fol_0000100` the trail happened to contain the branch decision $n = 1$ (part of an abandoned search region that was already known to be contradictory), and every endgame query inherited it — so every query answered `unsat` even though the discovered cells could perfectly well decompose a valid $\vec{v}$ with, say, $n = 4$. Level-0 facts are exactly those every model must satisfy: the query becomes "can the current cells justify *some* valid $\vec{v}$," which is the right question.

Why a fresh solver? The main solver could in principle answer the same question — Fact 3 shows why it never does: the question keeps being rewritten under it. This is the decoupling lesson from the failed main-solver experiment, applied in the profitable direction.

5.3 Part 3: feeding the witness back as decisions

The witness must now reach the *main* solver. Encoding it as a lemma $d \Rightarrow (x_1 = c_1 \wedge \cdots \wedge x_m = c_m)$, with d a fresh Boolean whose phase is preferred *true*, measurably never works.

Example (why the hint lemma never fires). Hint: $d \Rightarrow (a = 2 \wedge b = 3)$, i.e. clauses $(\neg d \vee a{=}2)$ and $(\neg d \vee b{=}3)$. Suppose the trail already contains $a = 1$ — some earlier decision or propagation made for unrelated reasons. Then the atom $a{=}2$ is *false* and the first clause *unit-propagates* $\neg d$. The phase preference on d is irrelevant: d is never *decided*, it is forced. With $m \approx 230$ pinned values, *some* variable is always already bound differently, so this happens every single round — measured: the hint guard was *false* at every check, forever.

The fix uses a different solver facility: a *decision strategy*. SAT solvers let theory modules dictate the next decision variable(s), taking priority over the built-in heuristic. While a hint is active, our strategy feeds the solver d first and then each equality $x_i = c_i$, as *decisions*. A decision happens at the top of the trail — nothing is "already bound differently" there; the solver either completes the witness assignment (and the model-value shortcut then certifies `sat`, since the witness satisfies $star_k$), or hits a genuine conflict whose reason is recorded and search continues. Only one hint is active per literal — a newer witness retires the previous guard with $\neg d_{old}$ — because two witnesses pin the same variables to different values and would fight.

The hint lemma itself is sound for the usual reason: d is fresh, so any model extends by $d = \mathit{false}$.

Measured result: `fol_0000100` went from a 600-second timeout (in every prior configuration) to `sat` in 0.7 seconds, model-checked; the whole hard `sat` family followed; 476/480 overall, with four `unsat` instances left.

6 Hard unsatisfiable instances: over-approximation by cuts

Four benchmarks remained, all `unsat`. This section explains, step by step, how they are refuted. We first build the intuition with pictures, then walk through the algorithm role by role, then show how it plays out on the real benchmarks, and finally describe the implementation inside `cvc5`.

6.1 Step 0: why these instances need something new

Everything before this section refutes ℓ in only one way: discover *all* cells, emit the exact equivalence $\ell \Leftrightarrow \mathit{star}$, and let arithmetic find the contradiction. On these four instances the predicate has astronomically many cells (the eager mode, which builds them all, also times out), so "finish the enumeration" is not a plan.

But look at what an unsat instance actually *is*: the input demands $\vec{v} \in S^*$ together with side constraints that no sum can meet. To expose the contradiction we do not need to know S^* exactly — we only need *one true fact about all of* S^* that the side constraints violate. A fact true of a superset is called an *over-approximation*; the rest of this section is about finding such facts automatically, in the form of linear inequalities we call *cuts*.

6.2 Step 1: the intuition — a fence through the origin

Mini example (used for the next three subsections). Let the predicate have a single point, $S' = \{(1,1)\}$. Then $(S')^* = \{(0,0),(1,1),(2,2),\ldots\}$ — the diagonal. Let the input assert ℓ and $v_1 = 3 \wedge v_2 = 5$. It is unsat: $(3,5)$ is not on the diagonal.

Figure 1 shows the geometry. The line $x - y = 0$ is a *fence through the origin*: every point of $(S')^*$ is on (or below) it, and the input point $(3,5)$ is strictly above it. So the single inequality $v_1 - v_2 \geq 0$ — one lemma — contradicts $v_1 = 3 \wedge v_2 = 5$, and the solver answers unsat having never enumerated anything.

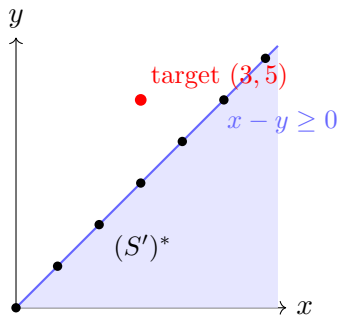


Figure 1: A cut is a fence through the origin: all of the star set on one side (shaded), the input's vector on the other. The fence must pass through the origin because $\vec{0}$ (the empty sum) is always in the star set.

Why must the fence pass through the origin, i.e. why must the inequality be *homogeneous* ($\vec{c} \cdot \vec{y} \geq 0$, no constant term)? Because that is exactly the shape that survives addition:

Proposition 2. *If* $\vec{c} \cdot \vec{y} \geq 0$ *for every* $\vec{y} \in S$*, then* $\vec{c} \cdot \vec{v} \geq 0$ *for every* $\vec{v} \in S^*$*.*

Proof. $\vec{v} = \vec{s}_1 + \cdots + \vec{s}_k$ gives $\vec{c} \cdot \vec{v} = \sum_j \vec{c} \cdot \vec{s}_j$, a sum of non-negative numbers; the empty sum gives $\vec{c} \cdot \vec{0} = 0 \geq 0$. □

Counter-example (a non-homogeneous fact does not lift). On $S' = \{(1,1)\}$ the inequality $x \leq 1$ is also true of every member — but $(2,2) \in (S')^*$ violates it. Adding two summands doubled the left side while the constant 1 stayed put. Only constant-free inequalities scale with the sum. (Same trap in the running example $p(x,y) = (x{\geq}1 \wedge y{=}0) \vee (x{=}0 \wedge y{\geq}2)$: every point of S satisfies $x + y \geq 1$, but the empty sum $\vec{0} \in S^*$ does not.)

So a cut is: *a homogeneous inequality, true of every point of the predicate, asserted for* $\vec{v}$ *as an ordinary unconditional lemma.* If we find cuts that together contradict the input's side constraints, the main solver concludes unsat by itself — cuts are just lemmas, so no new machinery is needed to "report" anything.

Two questions remain, and they are the heart of the matter:

1. Which inequalities are *allowed*? (True of all of S — but S has astronomically many cells, so how do we check that?)

2. Which inequalities are *useful*? (There are infinitely many valid cuts — $2x + 3y \geq 0$ is valid for every non-negative set — and almost all of them cut nothing the input cares about.)

The synthesis loop of Step 3 answers both with one solver query each.

6.3 Step 2: conditional cuts — using the zeros the input fixes

One refinement is needed before the loop, because on the multiset benchmarks no *globally* valid cut is strong enough.

Proposition 3. *Suppose the input forces $v_i = 0$. Since summands are non-negative and sums never cancel,* every *summand of every decomposition of $\vec{v}$ has $y_i = 0$. Hence an inequality that is valid only on the* restriction $S \cap \{y_i = 0\}$ *is still a sound cut for this particular $\vec{v}$.*

Example. Running example, with the input forcing $v_2 = 0$. Then every summand has $y = 0$, i.e. comes from the horizontal cell $\{(x, 0) \mid x \geq 1\}$. On that restriction, $x - y \geq 0$ is valid — even though it is *false* on the vertical cell $(0, 2), (0, 3), \ldots$ of the full predicate. Because the input itself rules the vertical cell out of every decomposition, the cut may be used.

Detecting forced zeros is free: input equalities like $u_{13} = 0$ are substituted into the literal during preprocessing, so the corresponding argument of ℓ is *literally the constant* 0 — the implementation just looks at the literal's arguments. All validity checks below then carry the extra assumptions $y_i = 0$ for those coordinates.

6.4 Step 3: the synthesis loop — target, candidate, validate

We now need to *find* a refuting cut automatically. The search space is the integer coefficient vector $\vec{c}$; what makes the problem interesting is that checking "$\vec{c}$ is valid on all of S" is itself a solver query. The loop, drawn in Figure 2, has three roles. Think of it as a game between a *prosecutor*, a *tailor*, and a *judge*.

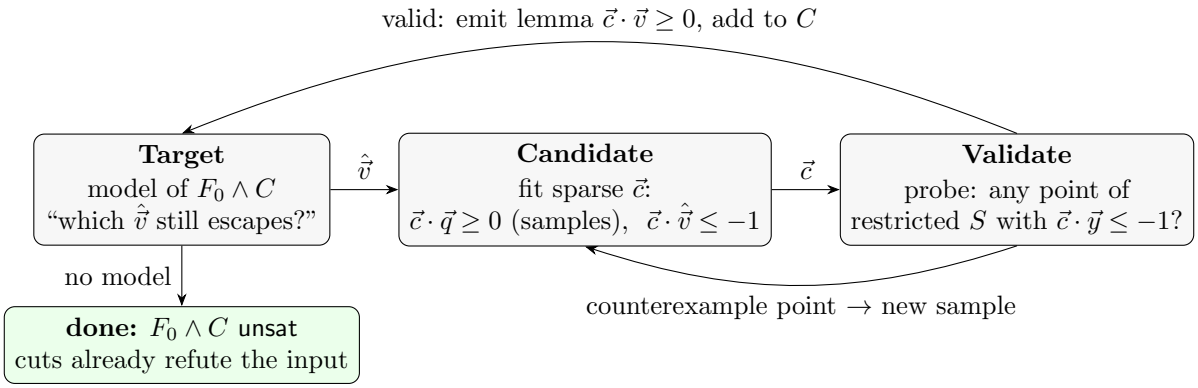


Figure 2: The cut synthesis loop. F_0 = input-entailed facts, C = cuts found so far, samples = known points of the (restricted) predicate.

Target (the prosecutor): *who must be cut?* Ask a small solver for a model of $F_0 \wedge C$: the facts the input fixes, plus the cuts already found. Its $\vec{v}$-values $\hat{\vec{v}}$ are a concrete vector the input still considers possible. Intuition: there is no point inventing inequalities at random — aim every new cut at a witness the input can still produce. Two outcomes:

- No model: $F_0 \wedge C$ is unsat — **the cuts already contradict the input**. Synthesis stops; the main solver, which has the same cut lemmas, derives unsat on its own.
- A model: its values $\hat{\vec{v}}$ become the target of the next cut.

In the mini example, before any cuts the target is forced: $\hat{\vec{v}} = (3, 5)$.

Candidate (the tailor): *guess a fence from the evidence so far.* Ask another small solver for integer coefficients $\vec{c}$ such that

$$\underbrace{\textstyle\sum_i |c_i| \le K}_{\text{simplicity budget}} \qquad \underbrace{\vec{c}\cdot\vec{q} \ge 0 \;\text{ for every known sample } \vec{q}}_{\text{respect the evidence}} \qquad \underbrace{\vec{c}\cdot\hat{\vec{v}} \le -1}_{\text{reject the target}} \;.$$

The *samples* are points known to lie in the (restricted) predicate — where they come from is part of the implementation, Step 6. The constraint $\vec{c}\cdot\hat{\vec{v}} \le -1$ uses -1 rather than < 0 because everything is integer: it is the same as "strictly negative", and it guarantees the finished cut genuinely excludes the target. Intuition: the tailor fits the simplest fence that keeps all known sheep on the safe side and the wolf outside. The tailor knows nothing about the predicate — only the finite evidence list. That ignorance is the point: fitting against finitely many constraints is a tiny, fast query.

Validate (the judge): *check the guess against the real predicate.* The candidate was only checked against samples; now ask the *probe* — a solver holding the actual predicate $base(\vec{y})$ — whether some point of the restricted predicate violates the candidate:

$$base(\vec{y}) \;\wedge\; \textstyle\bigwedge_{v_i \equiv 0} y_i = 0 \;\wedge\; \vec{c}\cdot\vec{y} \le -1 \qquad \stackrel{?}{=} \qquad \mathsf{sat}.$$

- unsat: no point of the predicate is on the wrong side — the candidate is *valid*. Emit the lemma $\vec{c}\cdot\vec{v} \ge 0$ and return to the prosecutor.
- sat: the model is a concrete predicate point on the wrong side of the fence — the guess was wrong, and here is the proof. Add the point to the samples and let the tailor try again. The same mistake is now impossible: the new sample constraint excludes every candidate that misclassifies this point.

This propose-from-finite-evidence / refute-with-a-counterexample cycle is the standard technique called CEGIS (counterexample-guided inductive synthesis). It terminates: for a fixed budget K the candidate space is finite, and every counterexample permanently eliminates at least the current candidate; every emitted cut permanently eliminates the current target.

6.5 Step 4: a complete run, with pictures

Worked run on the mini example ($S' = \{(1, 1)\}$, input $v_1 = 3 \wedge v_2 = 5$, no samples yet, budget $K = 4$).

Round 1 — Target: model of F_0 (no cuts yet) gives $\hat{\vec{v}} = (3, 5)$.

Round 1 — Candidate: with no samples, the only constraints are $|c_1| + |c_2| \le 4$ and $3c_1 + 5c_2 \le -1$. The solver may return $\vec{c} = (0, -1)$, the fence "$-y \ge 0$" (Figure 3, left). It

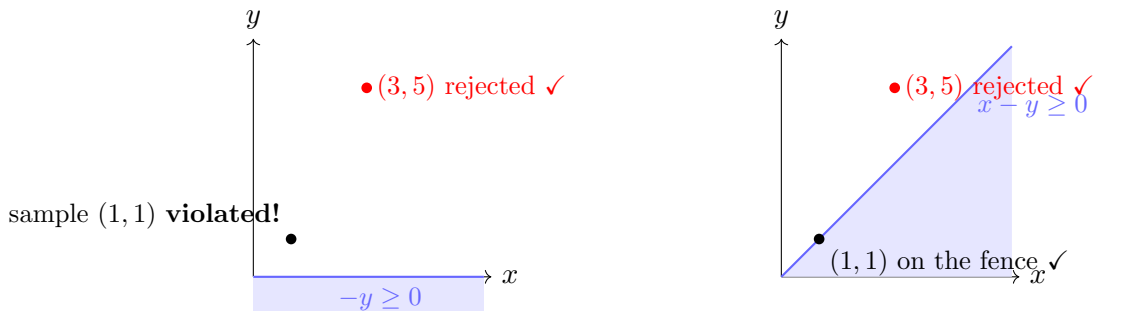
does reject the target ($-5 \le -1$).

Round 1 — Validate: is there a point of S' with $-y \le -1$? Yes: $(1,1)$. The guess put a sheep outside the fence. Add $(1,1)$ to the samples.

Round 2 — Candidate: now also require $c_1 \cdot 1 + c_2 \cdot 1 \ge 0$. Together with $3c_1 + 5c_2 \le -1$ this forces $c_1 \ge -c_2$ and pushes toward $\vec{c} = (1,-1)$: the fence $x - y \ge 0$ (Figure 3, right).

Round 2 — Validate: a point of S' with $x - y \le -1$? No — unsat. Valid! Emit the lemma $v_1 - v_2 \ge 0$.

Round 3 — Target: model of $F_0 \wedge (v_1 - v_2 \ge 0)$, i.e. $3 - 5 \ge 0$? unsat — done. The input is refuted by one cut.



(a) first guess: invalid

(b) refit after the counterexample: valid

Figure 3: One CEGIS refinement on the mini example. The first candidate separates the target but misclassifies a real predicate point; the counterexample becomes a sample, and the refit candidate is valid.

6.6 Step 5: why the budget ladder $K \in \{4, 9, 16, 30\}$

The budget bounds $\sum_i |c_i|$, the total weight of the inequality — informally, how *complicated* the fence is allowed to be. Examples: a pairwise comparison $y_i \le y_j$, i.e. $-y_i + y_j \ge 0$, has weight 2; the mini example's cut has weight 2; the real benchmark cut shown in Step 7 below has weight 9.

Why this step is needed. Without the budget, the search measurably diverges. The benchmark vectors have 21 coordinates; with coefficients in $[-10, 10]$ the candidate space has about $21^{21} \approx 10^{27}$ members. An unconstrained candidate solver returns *arbitrary dense* vectors — typically with a dozen nonzero coefficients — and each is refuted by one counterexample at a time; after 40 rounds the in-solver search had made no visible progress. The cuts that actually exist in these problems are *sparse*: they express simple counting facts over a handful of coordinates. Bounding the weight makes the candidate space tiny (weight ≤ 4 over 21 coordinates is a few thousand shapes), so each counterexample eliminates a meaningful fraction of it. With the ladder in place, the same loop refutes the hardest instance in 8 cuts and ∼33 candidate–validate rounds total.

Why these four rungs specifically? The choice is heuristic, and three properties matter more than the exact numbers:

- *Start small.* Most useful cuts are very light (weight 2–4): pairwise comparisons and small sums. A small first rung finds them almost instantly and keeps every later query cheap.

- *Cover the known need.* The heaviest cut we ever derived by hand on these benchmarks (the multiset cut of Step 7) has weight $4+1+1+1+1+1=9$ — the second rung is set exactly there.
- *Cap it.* Rungs 16 and 30 are headroom (roughly doubling each step, like the squares $2^2, 3^2, 4^2$ and a final allowance); the cap guarantees each synthesis attempt terminates. If no cut of weight ≤ 30 exists, the attempt is abandoned (and retried only when new information — new sample points from newly discovered cells — arrives).

Escalation is cheap because the samples *persist* across rungs and across synthesis attempts: a counterexample collected at $K=4$ still constrains the search at $K=9$.

6.7 Step 6: how it is implemented in cvc5

The synthesis is triggered from the endgame: whenever the decoupled check $F_0 \wedge star_k$ answers `unsat` ("the cells found so far cannot justify ℓ — and perhaps nothing can"), the extension calls `synthesizeCuts`. Everything is built from ordinary solver instances:

solver instance	lifetime	holds	question it answers
main solver	whole run	input + all lemmas	is the input satisfiable?
endgame	fresh per check	$F_0 \wedge star_k$	can the current cells justify ℓ?
target	fresh per round	$F_0 \wedge C$	which $\hat{\vec{v}}$ still escapes the cuts?
candidate	fresh per cut search	budget, samples, target	propose a sparse $\vec{c}$
validity probe	persistent per predicate	$base(\vec{y})$	is $\vec{c}\cdot\vec{y} \geq 0$ on the restricted predicate?

Concrete details, in the order they matter:

- *The facts F_0.* Taken from the arithmetic solver's asserted facts, keeping only those *fixed at decision level 0* (recall: facts holding in every model). Using the whole trail instead poisons the target: in one measured run a branch decision had pinned $n=1$ inside an already-doomed region, and every synthesis chased that phantom. Facts mentioning the star's internal unknowns (multipliers) and the star literals themselves are filtered out.
- *The candidate query.* Coefficient unknowns $c_1..c_n$ plus helper unknowns a_i with $a_i \geq c_i$ and $a_i \geq -c_i$ (so $a_i \geq |c_i|$), and $\sum_i a_i \leq K$ — the standard linear encoding of the weight budget. One linear constraint per sample, one for the target. The solver instance is incremental, so each counterexample is added as one new constraint.
- *The validity probe.* One persistent solver per predicate, with $base(\vec{y})$ asserted once (the same probe built for semantic generalization). Each validity check passes $\vec{c}\cdot\vec{y} \leq -1$ *and* the forced-zero equations $y_i = 0$ as *assumptions* — nothing is permanently asserted, so one probe serves every candidate, every budget rung, and every literal, whatever its zero pattern.
- *Seeding the samples.* Every cell already discovered by the lazy loop was processed by Normaliz, whose module generators are concrete points of S. Those lying in the restriction (zeros at the forced coordinates) are entered as samples up front, saving validate-rounds. **Pitfall (a real bug):** the cells' *ray directions* must *not* be used as samples. A ray of a cell pinned at $u_{13}=1$ has a 0 in the u_{13} coordinate — directions never change pinned equalities — so it passes the zero filter while describing motion inside a cell *entirely outside* the restriction. Constraining candidates by such rays excluded exactly the valid conditional cut on the hardest instance: an

offline replica of the loop *without* rays converged while the in-solver version *with* rays failed, which is how the bug was found. Far-out violations are still caught soundly — the probe just returns a far-out *point* as the counterexample.

- *Budgets that keep the common case fast.* At most 8 cuts per endgame call; at most 60 candidate–validate rounds per rung; after two consecutive failed syntheses the loop stops trying and resumes only when new cells (hence new samples) arrive. On `sat` instances the endgame usually answers `sat` before any synthesis is attempted, so the machinery costs nothing there.
- *Emission.* A valid cut is sent as the unconditional lemma $\vec{c} \cdot \vec{v} \geq 0$, tagged with its own inference identifier (`ARITH_LIA_STAR_CUT`); trivially-constant lemmas are skipped. Because cuts are plain lemmas, soundness needs no new argument beyond the two propositions above, and `unsat` is produced by the main solver's ordinary conflict machinery.

6.8 Step 7: what happens on the real benchmarks

The real predicates are counting structures: each summand carries indicator bits and magnitudes, and the input demands totals. The `unsat` arguments are budget arguments, and they are exactly homogeneous cuts. A miniature with the precise shape of `fol_0000098`:

Miniature of the real refutation. Let each summand be (cnt, A, B) with predicate

$$cnt = 1 \ \wedge \ A, B \in \{0,1\} \ \wedge \ \neg(A = 1 \wedge B = 1)$$

("each summand is one object that may carry badge A or badge B, but not both"). Every predicate point satisfies the weight-3 homogeneous inequality

$$cnt - A - B \ \geq \ 0 \qquad \text{(check: } (1,1,0) \mapsto 0,\ (1,0,1) \mapsto 0,\ (1,0,0) \mapsto 1\text{)}.$$

Summing over any decomposition: $v_{cnt} - v_A - v_B \geq 0$, i.e. *badges never outnumber objects.* Now let the input demand $v_{cnt} = n,\ v_A = n,\ v_B = 1$ ("n objects, all carrying A, and at least one B"). The cut gives $n - n - 1 \geq 0$: contradiction, `unsat` — found without looking at a single cell decomposition of the input. `fol_0000098` is this pattern with five badges and thresholds like $2v_A \geq n + 3t + 1$; the loop finds the needed cuts in well under a second.

The multiset variant `fol_0000107` additionally needs the conditional mechanism of Step 2: its refuting cut

$$4\,u_{10} - u_{14} - u_{17} - u_{20} - u_{23} - u_{26} \ \geq \ 0 \qquad \text{(weight 9)}$$

("the five tracked totals never exceed four universes") is valid *only* on the restriction where six comparison-indicator coordinates are 0 — which the input forces, since it pins exactly those coordinates of $\vec{v}$ to the constant 0. With conditional validity this cut is found and the instance refutes in 2 seconds; without it, the instance is out of reach.

Measured result: all four remaining benchmarks refute in 0.2–2.0 seconds, completing the suite at 480/480; no other instance is affected.

7 Results

All experiments use a production build of cvc5 with Normaliz, on 480 benchmarks (arith/card × BAPA/MAPA encodings of set-reachability problems); 100 s timeout; 12-way parallel harness; one

solver thread per instance; 6 GB memory cap per process. No answer discrepancy was observed anywhere: neither between any two of our configurations, nor against the four independent reference solvers recorded with the benchmark data.

Table 1: Solved instances per configuration (480 benchmarks, 100 s).

configuration	sat	unsat	timeout	solved
old recorded lazy baseline	272	178	30	450
lazy accumulate	283	180	17	463
lazy push/pop	280	180	20	460
lazy main-solver enumeration	273	174	33	447
lazy + boolean generalization	285	181	14	466
lazy + semantic generalization	284	181	15	465
lazy + partial sums	272	178	30	450
guided + endgame (no cuts)	296	180	4	476
final default (+ cuts)	**296**	**184**	**0**	**480**

The final default — boolean generalization, guided enumeration, the decoupled endgame with decision-strategy witness feedback, and conditional homogeneous cuts — solves the entire suite in 143.5 s cumulative (median 45 ms per instance, slowest instance 37.2 s), including multiset instances on which the reference solver that generated the benchmarks itself timed out.

option	default	role
`arith-liastar-generalize`	on	prime-implicant cells
`arith-liastar-guided`	on	decomposition-guided discovery
`arith-liastar-endgame=N`	10	decoupled witness search
`arith-liastar-cuts`	on	conditional homogeneous cuts
`arith-liastar-generalize-semantic`	off	arithmetic cell fattening
`arith-liastar-push-pop`	off	cumulative subsolver refinement
`arith-liastar-main-solver`	off	subsolver-free enumeration
`arith-liastar-partial-sums`	off	constant-size reduction lemmas
`arith-liastar-patient`	off	refinement-throttling experiment

8 Takeaways

- **Measure, then optimize.** The two memory-motivated designs (push/pop, partial sums) attacked a bottleneck that profiling refuted; the real walls were the *order* of cell discovery (sat side) and the missing over-approximation (unsat side).
- **Approximate from both sides.** Under-approximations certify sat as soon as enough cells exist; homogeneous (conditionally valid) cuts certify unsat without ever completing the enumeration. The lazy star needs both.
- **Decouple searches that starve each other.** In-search refinement kept rewriting the main solver's integer program; a fresh subsolver answers the same question in milliseconds. And when feeding a many-variable witness back, use a decision strategy — propagation defeats phase preferences every time.

- **Negative results pay rent.** The probe oracle built for the losing semantic generalization became the validity engine of the winning cut synthesis; the failed main-solver experiment supplied the decoupling principle behind the endgame.